PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Лабораторная работа №4. Написание Unit-тестов.

Смирнов Артемий Дмитриевич, М3119

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Для работы использовал библиотеку геометрических вычислений. Она содержит функции для вычисления площади и периметра круга, прямоугольника, квадрата и треугольника.

План тестирования библиотеки геометрических вычислений:

1. Цели и задачи тестирования

Цель тестирования — проверить правильность работы функций библиотеки geometric lib, которые вычисляют площадь и периметр геометрических фигур.

Задачи тестирования:

• подобрать входные значения для проверки функций;
• рассчитать ожидаемые результаты для выбранных значений;
• написать unit-тесты с использованием модуля unittest;
• запустить тесты и сравнить результаты работы функций с ожидаемыми значениями.

2. Описание тестируемого продукта

Geometric lib — библиотека, написанная на языке Python. Она предназначена для вычисления площади и периметра круга, прямоугольника, квадрата и треугольника. Функции получают размеры фигуры и возвращают вычисленное значение площади или периметра.

3. Область тестирования

В ходе тестирования проверяются четыре модуля библиотеки: rectangle, square, triangle и circle. В каждом модуле тестируются две функции: area для вычисления площади и perimeter для вычисления периметра фигуры. Для круга функция perimeter вычисляет длину окружности.

Проверяются вычисления функций. Проверка корректности размеров и существования треугольника не входит в область тестирования: исходные функции её не выполняют.

4. Стратегия тестирования

Для проверки отдельных функций используется модульное тестирование с помощью unittest. Для каждой функции подобраны целые, дробные, нулевые и положительные значения, близкие к нулю.

Ожидаемые результаты рассчитываются по формулам площади и периметра. Затем они сравниваются с результатами функций: для точного сравнения используется assertEqual, а для дробных результатов — assertAlmostEqual, который учитывает небольшую погрешность вычислений.

5. Критерии приёмки

Тестирование считается успешно завершённым, если все 37 тестов выполняются без ошибок и неуспешных проверок. Результаты функций должны совпадать с ожидаемыми значениями для выбранных входных данных; для дробных результатов допускается небольшая погрешность сравнения. По итогам запуска unittest должен вывести статус OK.

Успешное прохождение подтверждает работу функций на проверенных данных, но не доказывает отсутствие ошибок для всех возможных значений.

6. Ожидаемые результаты

PDF-XChange Editor
Click to BUY NOW!
www.pdf-xchange.com

По результатам тестирования ожидается получить результаты запуска unit-тестов: количество выполненных тестов и их общий статус. Если все проверки пройдут успешно, unittest выведет OK. Если тест не пройдёт, необходимо зафиксировать название теста и проверяемой функции, входные данные, ожидаемый и фактический результаты. Эти сведения помогут разобраться в причине ошибки.

```python
import unittest
import math
from circle import area, perimeter
class CircleTestCase(unittest.TestCase):  student2772737

    def test_area_normal(self):  student2772737
        result = area(3)
        self.assertAlmostEqual(result, 9 * math.pi)
    def test_area_zero(self):  student2772737
        result = area(0)
        self.assertEqual(result, 0)
    def test_area_float(self):  student2772737
        result = area(2.5)
        self.assertAlmostEqual(result, 6.25 * math.pi)
    def test_area_close_to_zero(self):  student2772737
        result = area(0.1)
        self.assertAlmostEqual(result, 0.01 * math.pi)

    def test_perimeter_normal(self):  student2772737
        result = perimeter(5)
        self.assertAlmostEqual(result, 10 * math.pi)
    def test_perimeter_zero(self):  student2772737
        result = perimeter(0)
        self.assertEqual(result, 0)
    def test_perimeter_float1(self):  student2772737
        result = perimeter(5.4)
        self.assertAlmostEqual(result, 10.8 * math.pi)
    def test_perimeter_close_to_zero(self):  student2772737
        result = perimeter(0.1)
        self.assertAlmostEqual(result, 0.2 * math.pi)
```

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Для проверки функций создал файлы с unit-тестами. В каждом файле используется модуль unittest. Тесты находятся в классе, наследующемся от unittest.TestCase. Проверил функции на целых положительных и нулевых значениях, а также на положительных значениях, близких к нулю.

```
PS C:\Games\ISRPOlabs\itmolab4> py -m unittest discover -v
test_area_close_to_zero (test_circle.CircleTestCase.test_area_close_to_zero) ... ok
test_area_float (test_circle.CircleTestCase.test_area_float) ... ok
test_area_normal (test_circle.CircleTestCase.test_area_normal) ... ok
test_area_zero (test_circle.CircleTestCase.test_area_zero) ... ok
test_perimeter_close_to_zero (test_circle.CircleTestCase.test_perimeter_close_to_zero) ... ok
test_perimeter_float1 (test_circle.CircleTestCase.test_perimeter_float1) ... ok
test_perimeter_normal (test_circle.CircleTestCase.test_perimeter_normal) ... ok
test_perimeter_zero (test_circle.CircleTestCase.test_perimeter_zero) ... ok
test_area_close_to_zero (test_rectangle.RectangleTestCase.test_area_close_to_zero) ... ok
test_area_first_zero (test_rectangle.RectangleTestCase.test_area_first_zero) ... ok
test_area_float1 (test_rectangle.RectangleTestCase.test_area_float1) ... ok
test_area_float2 (test_rectangle.RectangleTestCase.test_area_float2) ... ok
test_area_normal (test_rectangle.RectangleTestCase.test_area_normal) ... ok
test_area_second_zero (test_rectangle.RectangleTestCase.test_area_second_zero) ... ok
test_perimeter_close_to_zero (test_rectangle.RectangleTestCase.test_perimeter_close_to_zero) ... ok
test_perimeter_first_zero (test_rectangle.RectangleTestCase.test_perimeter_first_zero) ... ok
test_perimeter_float1 (test_rectangle.RectangleTestCase.test_perimeter_float1) ... ok
test_perimeter_float2 (test_rectangle.RectangleTestCase.test_perimeter_float2) ... ok
test_perimeter_normal (test_rectangle.RectangleTestCase.test_perimeter_normal) ... ok
test_perimeter_second_zero (test_rectangle.RectangleTestCase.test_perimeter_second_zero) ... ok
test_area_close_to_zero (test_square.SquareTestCase.test_area_close_to_zero) ... ok
test_area_float (test_square.SquareTestCase.test_area_float) ... ok
test_area_normal (test_square.SquareTestCase.test_area_normal) ... ok
test_area_zero (test_square.SquareTestCase.test_area_zero) ... ok
test_perimeter_close_to_zero (test_square.SquareTestCase.test_perimeter_close_to_zero) ... ok
test_perimeter_float1 (test_square.SquareTestCase.test_perimeter_float1) ... ok
test_perimeter_normal (test_square.SquareTestCase.test_perimeter_normal) ... ok
test_perimeter_zero (test_square.SquareTestCase.test_perimeter_zero) ... ok
test_area_close_to_zero (test_triangle.TestTriangle.test_area_close_to_zero) ... ok
test_area_float (test_triangle.TestTriangle.test_area_float) ... ok
test_area_normal (test_triangle.TestTriangle.test_area_normal) ... ok
test_area_zero1 (test_triangle.TestTriangle.test_area_zero1) ... ok
test_area_zero2 (test_triangle.TestTriangle.test_area_zero2) ... ok
test_perimeter_close_to_zero (test_triangle.TestTriangle.test_perimeter_close_to_zero) ... ok
test_perimeter_float (test_triangle.TestTriangle.test_perimeter_float) ... ok
test_perimeter_normal (test_triangle.TestTriangle.test_perimeter_normal) ... ok
test_perimeter_zero (test_triangle.TestTriangle.test_perimeter_zero) ... ok

----------------------------------------------------------------------
Ran 37 tests in 0.007s

OK
```

Запустил все тесты командой py -m unittest discover -v. Выполнилось 37 тестов для функций площади и периметра четырёх фигур. Все тесты прошли успешно.

PDF-XChange Editor
Click to BUY NOW!
www.pdf-xchange.com

Для запуска всех тестов нужно открыть терминал в корневой папке проекта и вы[illegible]нить:

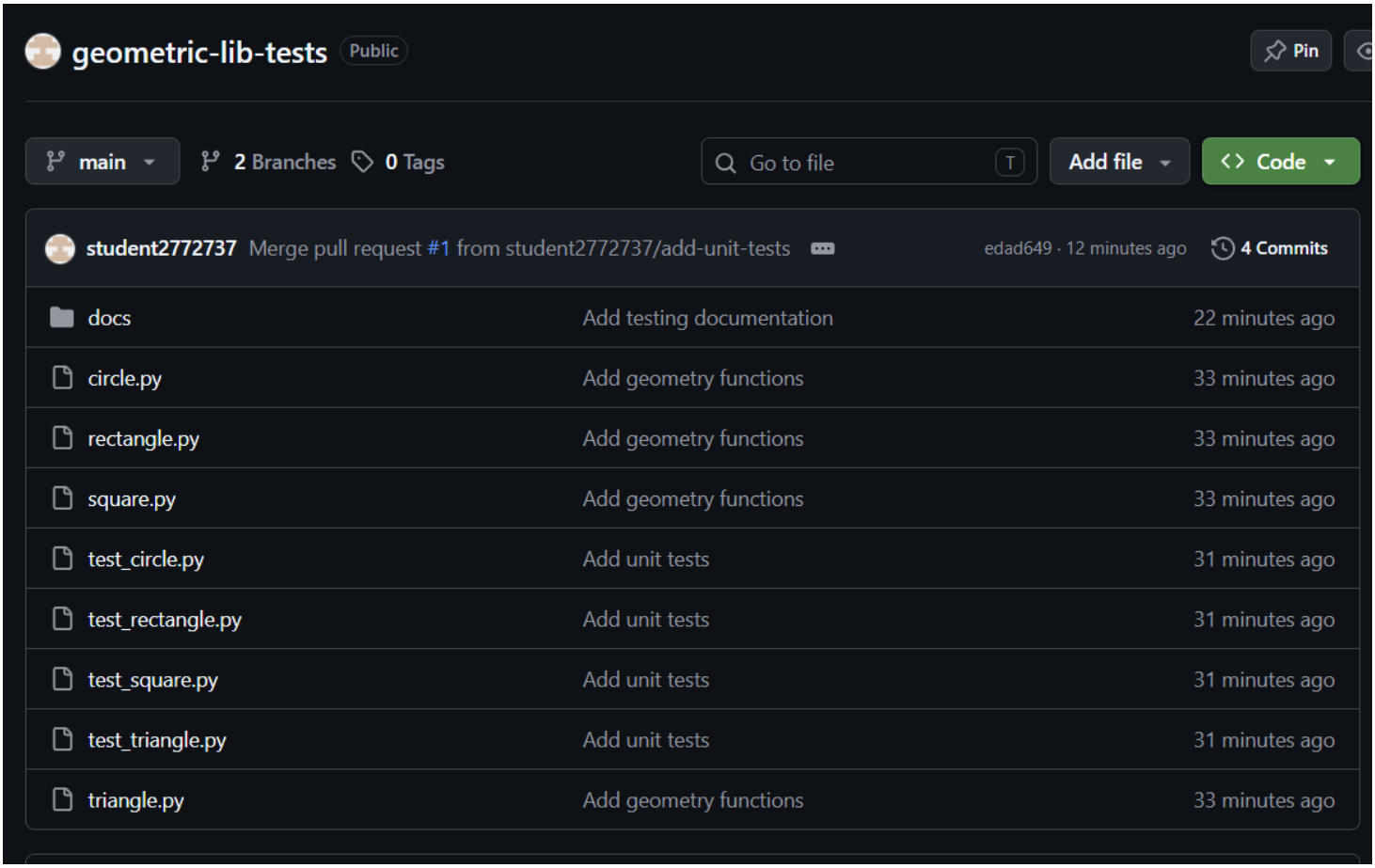
```
py -m unittest discover -v
```

При успешном выполнении всех тестов в конце вывода появится `OK`.

История изменений

Хеш коммита	Описание
`21c9f86`	Добавлены функции для геометрических вычислений
`f3a3a21`	Добавлены unit-тесты для функций площади и периметра

Составил план тестирования и добавил в документацию информацию о запуске тестов. В истории изменений указал коммит, в котором были добавлены unit-тесты.



Создал публичный репозиторий и загрузил в него исходные файлы, тесты и документацию. Тесты добавил в отдельной ветке add-unit-tests, затем объединил её с main через Pull Request.

https://github.com/student2772737/geometric-lib-tests

В ходе работы я научился определять цели тестирования, разрабатывать стратегию и критерии приёмки. Я написал и запустил unit-тесты для функций вычисления площади и периметра геометрических фигур. Все 37 тестов прошли успешно на выбранных входных данных.